

MODULE 2

2.6 BIVARIATE DATA AND MULTIVARIATE DATA

Bivariate Data involves two variables. Bivariate data deals with causes of relationships. The aim is to find relationships among data. Consider the following Table 2.3, with data of the temperature in a shop and sales of sweaters.

Table 2.3: Temperature in a Shop and Sales Data

Temperature (in centigrade)	Sales of Sweaters (in thousands)
5	200
10	150
15	140
20	75
22	60
23	55
25	20

Here, the aim of bivariate analysis is to find relationships among variables. The relationships can then be used in comparisons, finding causes, and in further explorations. To do that, graphical display of the data is necessary. One such graph method is called scatter plot.

Scatter plot is used to visualize bivariate data. It is useful to plot two variables with or without nominal variables, to illustrate the trends, and also to show differences. It is a plot between explanatory and response variables. It is a 2D graph showing the relationship between two variables.

The scatter plot (Refer Figure 2.11) indicates strength, shape, direction and the presence of Outliers. It is useful in exploratory data before calculating a correlation coefficient or fitting regression curve.

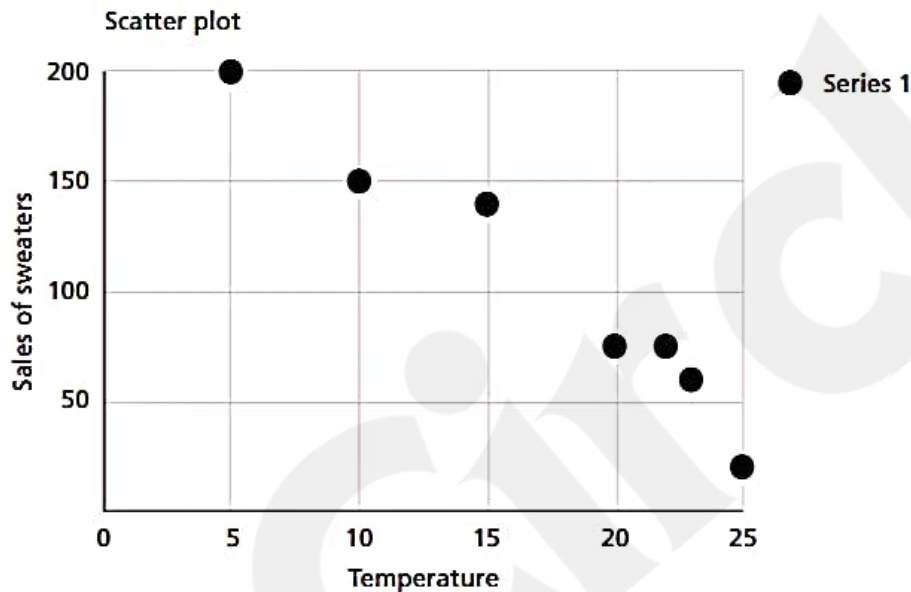


Figure 2.11: Scatter Plot

Line graphs are similar to scatter plots. The Line Chart for sales data is shown in Figure 2.12.

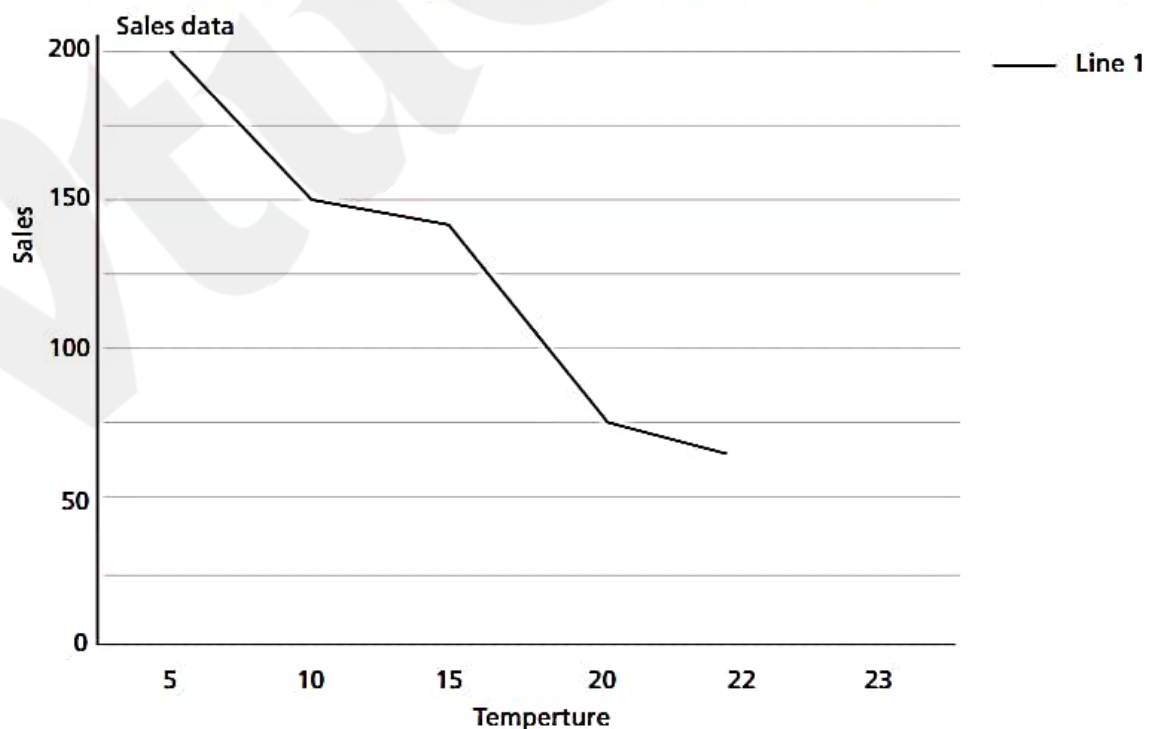


Figure 2.12: Line Chart

2.6.1 Bivariate Statistics

Covariance and Correlation are examples of bivariate statistics. Covariance is a measure of joint probability of random variables, say X and Y . Generally, random variables are represented in capital letters. It is defined as $\text{covariance}(X, Y)$ or $\text{COV}(X, Y)$ and is used to measure the variance between two dimensions. The formula for finding co-variance for specific x , and y are:

$$\text{cov}(X, Y) = \frac{1}{N} \sum_{i=1}^N (x_i - E(X))(y_i - E(Y)) \quad (2.17)$$

Here, x_i and y_i are data values from X and Y . $E(X)$ and $E(Y)$ are the mean values of x_i and y_i . N is the number of given data. Also, the $\text{COV}(X, Y)$ is same as $\text{COV}(Y, X)$.

Example 2.6: Find the covariance of data $X = \{1, 2, 3, 4, 5\}$ and $Y = \{1, 4, 9, 16, 25\}$.

Solution: $\text{Mean}(X) = E(X) = \frac{15}{5} = 3$, $\text{Mean}(Y) = E(Y) = \frac{55}{5} = 11$. The covariance is computed using Eq. (2.17) as:

$$\frac{(1-3)(1-11) + (2-3)(4-11) + (3-3)(9-11) + (4-3)(16-11) + (5-3)(25-11)}{5} = 12$$

The covariance between X and Y is 12. It can be normalized to a value between -1 and $+1$. This is done by dividing it by the correlation of variables. This is called Pearson correlation coefficient. Sometimes, $N - 1$ is also can be used instead of N . In that case, the covariance is $60/4 = 15$.

Correlation

The Pearson correlation coefficient is the most common test for determining any association between two phenomena. It measures the strength and direction of a *linear* relationship between the x and y variables.

The correlation indicates the relationship between dimensions using its sign. The sign is more important than the actual value.

1. If the value is positive, it indicates that the dimensions increase together.
2. If the value is negative, it indicates that while one-dimension increases, the other dimension decreases.
3. If the value is zero, then it indicates that both the dimensions are independent of each other.

If the dimensions are correlated, then it is better to remove one dimension as it is a redundant dimension.

If the given attributes are $X = (x_1, x_2, \dots, x_N)$ and $Y = (y_1, y_2, \dots, y_N)$, then the Pearson correlation coefficient, that is denoted as r , is given as:

$$r = \frac{\text{COV}(X, Y)}{\sigma_X \sigma_Y} \quad (2.18)$$

where, σ_X, σ_Y are the standard deviations of X and Y .

Example 2.7: Find the correlation coefficient of data $X = \{1, 2, 3, 4, 5\}$ and $Y = \{1, 4, 9, 16, 25\}$.

Solution: The mean values of X and Y are $\frac{15}{5} = 3$ and $\frac{55}{5} = 11$. The standard deviations of X and Y are 1.41 and 8.6486, respectively. Therefore, the correlation coefficient is given as ratio of covariance (12 from the previous problem 2.5) and standard deviation of x and y as per Eq. (2.18) as:

$$r = \frac{12}{1.41 \times 8.6486} \approx 0.984$$

2.7 MULTIVARIATE STATISTICS

In machine learning, almost all datasets are multivariable. Multivariate data is the analysis of more than two observable variables, and often, thousands of multiple measurements need to be conducted for one or more subjects.

The multivariate data is like bivariate data but may have more than two dependant variables. Some of the multivariate analysis are regression analysis, principal component analysis, and path analysis.

<i>Id</i>	<i>Attribute 1</i>	<i>Attribute 2</i>	<i>Attribute 3</i>
1	1	4	1
2	2	5	2
3	3	6	1

The mean of multivariate data is a mean vector and the mean of the above three attributes is given as (2, 7.5, 1.33). The variance of multivariate data becomes the covariance matrix. The mean vector is called centroid and variance is called dispersion matrix. This is discussed in the next section.

Multivariate data has three or more variables. The aim of the multivariate analysis is much more. They are regression analysis, factor analysis and multivariate analysis of variance that are explained in the subsequent chapters of this book.

Heatmap

Heatmap is a graphical representation of 2D matrix. It takes a matrix as input and colours it. The darker colours indicate very large values and lighter colours indicate smaller values. The advantage of this method is that humans perceive colours well. So, by colour shaping, larger values can be perceived well. For example, in vehicle traffic data, heavy traffic regions can be differentiated from low traffic regions through heatmap.

In Figure 2.13, patient data highlighting weight and health status is plotted. Here, X-axis is weights and Y-axis is patient counts. The dark colour regions highlight patients' weights vs patient counts in health status.

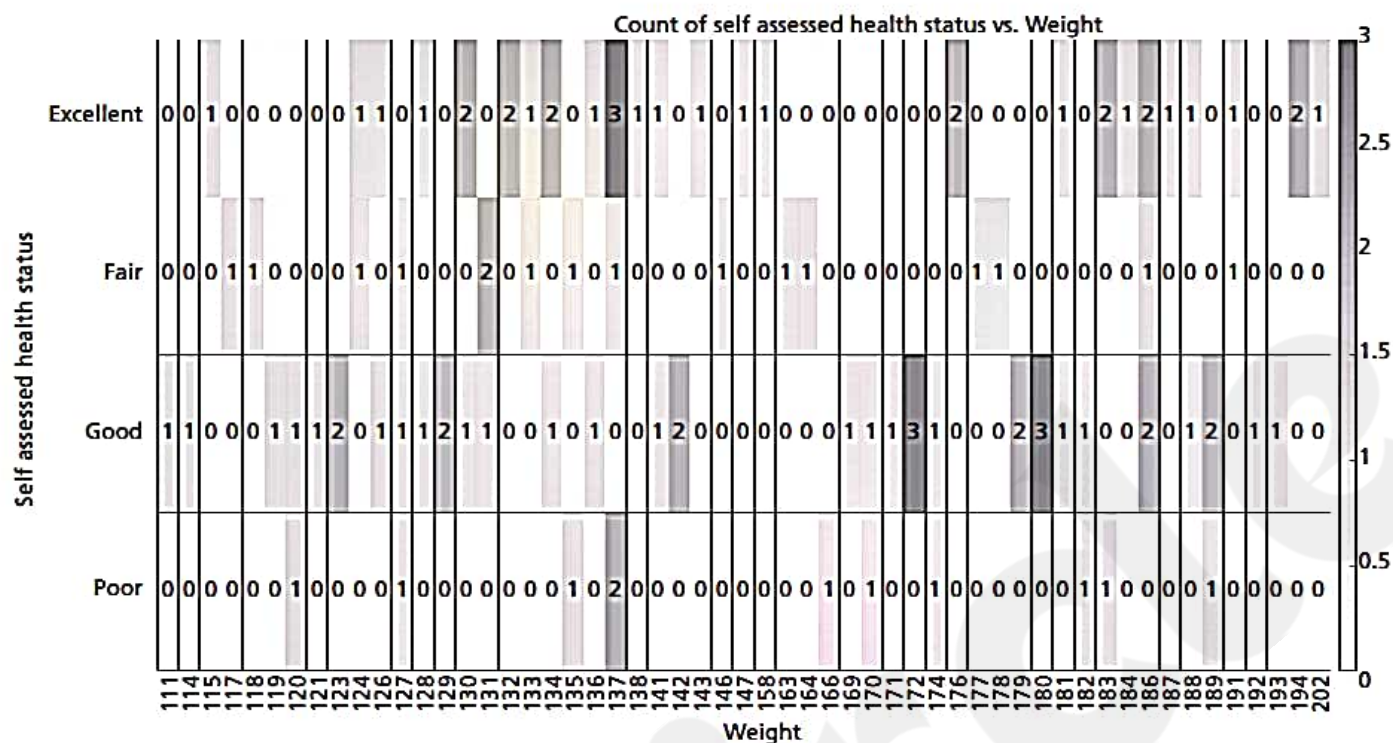


Figure 2.13: Heatmap for Patient Data

Pairplot

Pairplot or scatter matrix is a data visual technique for multivariate data. A scatter matrix consists of several pair-wise scatter plots of variables of the multivariate data. All the results are presented in a matrix format. By visual examination of the chart, one can easily find relationships among the variables such as correlation between the variables.

A random matrix of three columns is chosen and the relationships of the columns is plotted as a pairplot (or scattermatrix) as shown below in Figure 2.14.

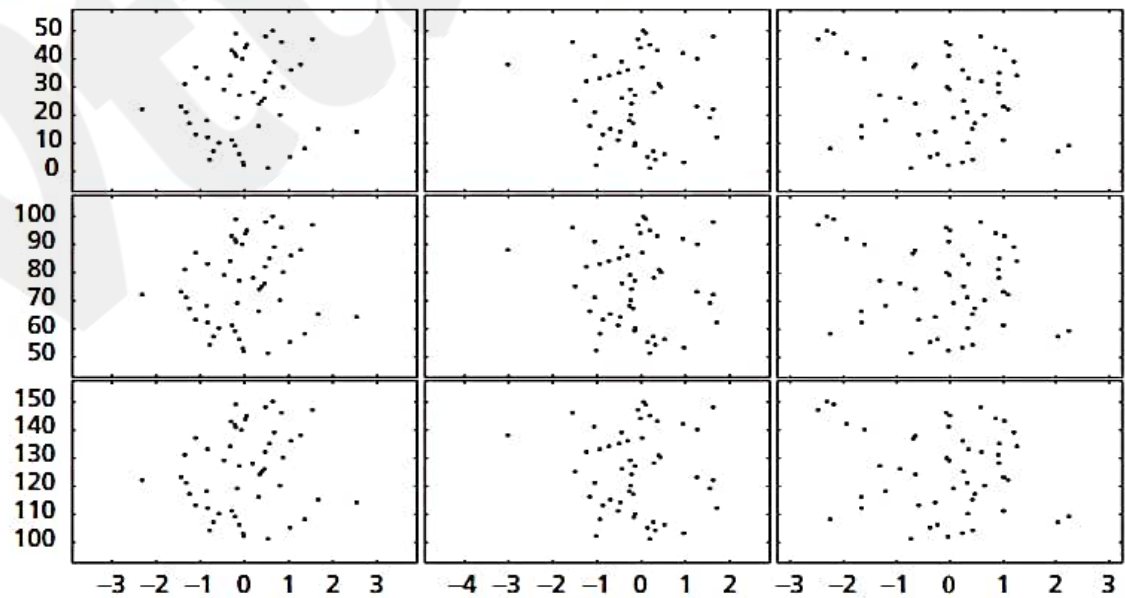


Figure 2.14: Pairplot for Random Data

2.8 ESSENTIAL MATHEMATICS FOR MULTIVARIATE DATA

Machine learning involves many mathematical concepts from the domain of Linear algebra, Statistics, Probability and Information theory. The subsequent sections discuss important aspects of linear algebra and probability.

'Linear Algebra' is a branch of mathematics that is central for many scientific applications and other mathematical subjects. While all branches of mathematics are crucial for machine learning, linear algebra plays a major large role as it is the mathematics of data. Linear algebra deals with linear equations, vectors, matrices, vector spaces and transformations. These are the driving forces of machine learning and machine learning cannot exist without these data types.

Let us discuss some of the important concepts of linear algebra now.

2.8.1 Linear Systems and Gaussian Elimination for Multivariate Data

A linear system of equations is a group of equations with unknown variables.

Let $Ax = y$, then the solution x is given as:

$$x = y/A = A^{-1} y \quad (2.19)$$

This is true if y is not zero and A is not zero. The logic can be extended for N -set of equations with ' n ' unknown variables.

It means if $A = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & \vdots & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{nn} \end{pmatrix}$ and $y = (y_1 \ y_2 \dots y_n)$, then the unknown variable x can be computed as:

$$x = y/A = A^{-1} y \quad (2.20)$$

If there is a unique solution, then the system is called consistent independent. If there are various solutions, then the system is called consistent dependant. If there are no solutions and if the equations are contradictory, then the system is called inconsistent.

For solving large number of system of equations, Gaussian elimination can be used. The procedure for applying Gaussian elimination is given as follows:

1. Write the given matrix.
2. Append vector y to the matrix A . This matrix is called augmentation matrix.
3. Keep the element a_{11} as pivot and eliminate all a_{1i} in second row using the matrix operation,

$R_2 - \left(\frac{a_{21}}{a_{11}} \right)$, here R_2 is the second row and $\left(\frac{a_{21}}{a_{11}} \right)$ is called the multiplier. The same logic can be used to remove a_{1i} in all other equations.

4. Repeat the same logic and reduce it to reduced echelon form. Then, the unknown variable as:

$$x_n = \frac{y_{nn}}{a_{nn}} \quad (2.21)$$

5. Then, the remaining unknown variables can be found by back-substitution as:

$$x_{n-1} = \frac{y_{n-1} - a_{n-1} \times x_n}{a_{(n-1)(n-1)}} \quad (2.22)$$

This part is called backward substitution.

To facilitate the application of Gaussian elimination method, the following row operations are applied:

1. Swapping the rows
2. Multiplying or dividing a row by a constant
3. Replacing a row by adding or subtracting a multiple of another row to it

These concepts are illustrated in Example 2.8.

Example 2.8: Solve the following set of equations using Gaussian Elimination method.

$$2x_1 + 4x_2 = 6$$

$$4x_1 + 3x_2 = 7$$

Solution: Rewrite this in matrix form as follows:

$$\begin{pmatrix} 2 & 4 & | & 6 \\ 4 & 3 & | & 7 \end{pmatrix} \\ \sim \begin{pmatrix} 2 & 4 & | & 6 \\ 4 & 3 & | & 7 \end{pmatrix} R_1 = \frac{R_1}{2}$$

Apply the transformation by dividing the row 1 by 2. There are no general guidelines of row operations other than reducing the given matrix to row echelon form. The operator $\sim$ means reducing to. The above matrix can further be reduced as follows:

$$\sim \begin{pmatrix} 1 & 2 & | & 3 \\ 4 & 3 & | & 7 \end{pmatrix} R_2 = R_2 - 4R_1 \\ \sim \begin{pmatrix} 1 & 2 & | & 3 \\ 0 & -5 & | & -5 \end{pmatrix} R_2 = R_2 / -5 \\ \sim \begin{pmatrix} 1 & 2 & | & 3 \\ 0 & 1 & | & 1 \end{pmatrix} R_1 = R_1 - 2R_2 \\ \sim \begin{pmatrix} 1 & 0 & | & 1 \\ 0 & 1 & | & 1 \end{pmatrix}$$

Therefore, in the reduced echelon form, it can be observed that:

$$x_2 = 1$$

$$x_1 = 1$$

2.8.2 Matrix Decompositions

It is often necessary to reduce a matrix to its constituent parts so that complex matrix operations can be performed. These methods are also known as matrix factorization methods.

The most popular matrix decomposition is called eigen decomposition. It is a way of reducing the matrix into eigen values and eigen vectors.

Then, the matrix A can be decomposed as:

$$A = Q \Lambda Q^T \quad (2.23)$$

where, Q is the matrix of eigen vectors, Λ is the diagonal matrix and Q^T is the transpose of matrix Q .

LU Decomposition

One of the simplest matrix decompositions is LU decomposition where the matrix A can be decomposed matrices:

$$A = LU$$

Here, L is the lower triangular matrix and U is the upper triangular matrix. The decomposition can be done using Gaussian elimination method as discussed in the previous section. First, an identity matrix is augmented to the given matrix. Then, row operations and Gaussian elimination is applied to reduce the given matrix to get matrices L and U .

Example 2.9 illustrates the application of Gaussian elimination to get LU .

Example 2.9: Find LU decomposition of the given matrix:

$$A = \begin{pmatrix} 1 & 2 & 4 \\ 3 & 3 & 2 \\ 3 & 4 & 2 \end{pmatrix}$$

Solution: First, augment an identity matrix and apply Gaussian elimination. The steps are as shown in:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 3 & 3 & 2 \\ 3 & 4 & 2 \end{bmatrix} \quad \boxed{\text{Initial Matrix}}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 3 & 4 & 2 \end{bmatrix} \quad \boxed{R_2 = R_2 - 3R_1}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 3 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 0 & -2 & -10 \end{bmatrix} \quad \boxed{R_3 = R_3 - 3R_1}$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 3 & \frac{2}{3} & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 0 & 0 & -\frac{10}{3} \end{bmatrix} \quad \boxed{R_3 = R_3 - \frac{2}{3}R_2}$$

Now, it can be observed that the first matrix is L as it is the lower triangular matrix whose values are the determiners used in the reduction of equations above such as 3, 3 and $2/3$. The second matrix is U , the upper triangular matrix whose values are the values of the reduced matrix because of Gaussian elimination.

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ 3 & \frac{2}{3} & 1 \end{pmatrix} \quad \text{and} \quad U = \begin{pmatrix} 1 & 2 & 4 \\ 0 & -3 & -10 \\ 0 & 0 & -\frac{10}{3} \end{pmatrix}$$

It can be cross verified that the multiplication of LU yields the original matrix A . What are the applications of LU decomposition? Some of the applications are finding matrix inverses and determinant. If the order of the matrix is large, then this method can be used.

2.8.3 Machine Learning and Importance of Probability and Statistics

Machine learning is linked with statistics and probability. Like linear algebra, statistics is the heart of machine learning. The importance of statistics needs to be stressed as without statistics; analysis of data is difficult. Probability is especially important for machine learning. Any data can be assumed to be generated by a probability distribution. Machine learning datasets have multiple data that are generated by multiple distributions. So, a knowledge of probability distribution and random variables are must for better understanding of the machine learning concepts.

What is the objective for an experiment? This is about hypothesis or constructing a model. Machine learning has many models. So, the theory of hypothesis, construction of models, evaluating the models using hypothesis testing, significance of the model and evaluation of the model are all linking factors of machine learning and probability theory. Similarly, the construction of datasets using sampling theory is also required.

The subsequent section will deal with the important concepts of statistics and probability.

Probability Distributions

A probability distribution of a variable, say X , summarizes the probability associated with X 's events. Distribution is a parameterized mathematical function. In other words, distribution is a function that describes the relationship between the observations in a sample space.

Consider a set of data. The data is said to follow a distribution if it obeys a mathematical function that characterizes that distribution. The function can be used to calculate the probability of individual observations.

Probability distributions are of two types:

1. Discrete probability distribution
2. Continuous probability distribution

The relationships between the events for a continuous random variable and their probabilities is called a continuous probability distribution. It is summarized as Probability Density Function (PDF). PDF calculates the probability of observing an instance. The plot of PDF shows the shape of the distribution. Cumulative Distributive Function (CDF) computes the probability of an observation $\leq$ value.

Both PDF and CDF are continuous values. The discrete equivalent of PDF in discrete distribution is called Probability Mass Function (PMF).

The probability of an event cannot be detected directly. It should be computed as the area under the curve for a small interval around the specific outcome. This is defined as CDF.

Let us discuss some of the distributions that are encountered in machine learning.

Continuous Probability Distributions Normal, Rectangular, and Exponential distributions fall under this category.

1. Normal Distribution – Normal distribution is a continuous probability distribution. This is also known as gaussian distribution or bell-shaped curve distribution. It is the most common distribution function. The shape of this distribution is a typical bell-shaped curve. In normal distribution, data tends to be around a central value with no bias on left or right. The heights of the students, blood pressure of a population, and marks scored in a class can be approximated using normal distribution.

PDF of the normal distribution is given as:

$$f(x, \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} \quad (2.24)$$

Here, μ is mean and σ is the standard deviation. Normal distribution is characterized by two parameters – mean and variance.

Mostly, one uses the normal distribution curve of mean 0 and a SD of 1. In normal distribution, mean, median and mode are same. The distribution extends from $-\infty$ to $+\infty$. Standard deviation is how the data is spread out.

One important concept associated with normal distribution is z-score. It can be computed as:

$z = \frac{x - \mu}{\sigma}$. When μ is zero and σ is 1, z-score is same as x . This is useful to normalize the data.

Most of the statistical tests expect data to follow normal distribution. To check it, normality tests are used. Normality test of the data can be done by Q-Q plot where CDF of one random variable follows CDF of normal distribution. Then, quantity of one distribution is plotted against other distributions. If they are same, then the plot closely follows the straight line from bottom-left to top-right.

2. Rectangular Distribution – This is also known as uniform distribution. It has equal probabilities for all values in the range a, b . The uniform distribution is given as follows:

$$P(X = x) = \begin{cases} \frac{1}{b - a} & \text{for } a \leq x \leq b \\ 0 & \text{Otherwise} \end{cases} \quad (2.25)$$

3. Exponential Distribution – This is a continuous uniform distribution. This probability distribution is used to describe the time between events in a Poisson process. Exponential distribution is another special case of Gamma distribution with a fixed parameter of 1. This distribution is helpful in modelling of time until an event occurs.

The PDF is given as follows:

$$f(x, \lambda) = \begin{cases} \lambda e^{-\lambda x} & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases} \quad (\lambda > 0) \quad (2.26)$$

Here, x is a random variable and λ is called rate parameter. The mean and standard deviation of exponential distribution is given as β , where, $\beta = \frac{1}{\lambda}$.

Discrete Distribution Binomial, Poisson, and Bernoulli distributions fall under this category.

1. **Binomial Distribution** – Binomial distribution is another distribution that is often encountered in machine learning. It has only two outcomes: success or failure. This is also called Bernoulli trial.

The objective of this distribution is to find probability of getting success k out of n trials. The way to get success out of k out of n number of trials is given as:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!} \quad (2.27)$$

The binomial distribution function is given as follows, where p is the probability of success and probability of failure is $(1 - p)$. The probability of success in a certain number of trials is given as:

$$p^k(1-p)^{n-k} \text{ or } p^k q^{n-k} \quad (2.28)$$

Combining both, one gets PDF of binomial distribution as:

$$\binom{n}{k} p^k (1-p)^{n-k} \quad (2.29)$$

Here, p is the probability of each choice, k is the number of choices, and n is the total number of choices. The mean of binomial distribution is given below:

$$\mu = n \times p \quad (2.30)$$

And the variance is given as:

$$\sigma^2 = np(1-p) \quad (2.31)$$

Hence, the standard deviation is given as:

$$\sigma = \sqrt{np(1-p)} \quad (2.32)$$

2. **Poisson Distribution** – It is another important distribution that is quite useful. Given an interval of time, this distribution is used to model the probability of a given number of events k . The mean rule λ is inclusive of previous events. Some of the examples of Poisson distribution are number of emails received, number of customers visiting a shop and the number of phone calls received by the office.

The PDF of Poisson distribution is given as follows:

$$f(X = x; \lambda) = Pr[X = x] = \frac{e^{-\lambda} \lambda^x}{x!} \quad (2.33)$$

Here, x is the number of times the event occurs and λ is the mean number of times an event occurs.

The mean is the population mean at number of emails received and the standard deviation is $\sqrt{\lambda}$.

3. **Bernoulli Distribution** – This distribution models an experiment whose outcome is binary. The outcome is positive with p and negative with $1 - p$. The PMF of this distribution is given as:

$$f(k; p) = \begin{cases} q = 1 - p & \text{if } k = 0 \\ p & \text{if } k = 1. \end{cases} \quad (2.34)$$

The mean is p and variance is $p(1-p) = q$

Density Estimation

Let there be a set of observed values $x_1, x_2, \dots, x_n$ from a larger set of data whose distribution is not known. Density estimation is the problem of estimating the density function from an observed data. The estimated density function, denoted as, $p(x)$ can be used to value directly for any unknown data, say x_i as $p(x_i)$. If its value is less than ϵ , then x_i is not an outlier or anomaly data. Else, it is categorized as an anomaly data.

There are two types of density estimation methods, namely parametric density estimation and non-parametric density estimation.

Parametric Density Estimation It assumes that the data is from a known probabilistic distribution and can be estimated as $p(x | \Theta)$, where, Θ is the parameter. Maximum likelihood function is a parametric estimation method.

Maximum Likelihood Estimation For a sample of observations, one can estimate the probability distribution. This is called density estimation. Maximum Likelihood Estimation (MLE) is a probabilistic framework that can be used for density estimation. This involves formulating a function called likelihood function which is the conditional probability of observing the observed samples and distribution function with its parameters. For example, if the observations are $X = \{x_1, x_2, \dots, x_n\}$, then density estimation is the problem of choosing a PDF with suitable parameters to describe the data. MLE treats this problem as a search or optimization problem where the probability should be maximized for the joint probabilities of X and its parameter, θ .

For example, this is expressed as $p(X; \theta)$, where, $X = \{x_1, x_2, \dots, x_n\}$

The likelihood of observing the data is given as a function $L(X; \theta)$. The objective of MLE is to maximize this function as $\max L(X; \theta)$.

The joint probability of this problem can be restated as $\prod_{i=1}^n p(x_i; \theta)$.

The computation of the above formula is unstable and the hence the problem is restated as maximum of log conditional probability given θ . This is given as:

$$\sum_{i=1}^n \log p(x_i; \theta) \quad (2.35)$$

Instead of maximizing, one can minimize this function as:

$$\min = -\sum_{i=1}^n \log p(x_i; \theta) \text{ as, often, minimization is preferred over maximization.}$$

This is called negative log-likelihood function.

The relevance of this theory of MLE for machine learning is that MLE can solve the problem of predictive modelling in machine learning. Regression problem is treated in Chapter 5 using the principle of least-square approach. The same problem can be viewed from MLE perspective too. If one assumes that the regression problem can be framed as predicting output y given input x , then for $p(y/x)$, the MLE framework can be applied as:

$$\max \sum \log(y | x_i, h) \quad (2.36)$$

Here, h is the linear regression model. If Gaussian distribution is assumed as it is an obvious fact that most of the data follow Gaussian distribution, then MLE can be stated as:

$$\max \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{y_i - h(x_i; \beta)}{2\sigma^2}} \quad (2.37)$$

Here, β is the regression coefficient and x_i is the given sample. One can maximize this function or minimize the negative log likelihood function to provide a solution for linear regression problem. The Eq. (2.37) yields the same answer of the least-square approach.

Stochastic gradient descent (SGD) is an algorithm that is normally used to maximize or minimize any function. This algorithm can be used to minimize the above function to get the results effectively. Hence, many statistical packages use this approach for solving linear regression problem. Theoretically, any model can be used instead of linear regression in the above equation. Logistic regression is another model that can be used in MLE framework. Logistic regression is discussed in Chapter 5 of this book.

Gaussian Mixture Model and Expectation-Maximization (EM) Algorithm In machine learning, clustering is one of the important tasks. It is discussed in Chapter 13. MLE framework is quite useful for designing model-based methods for clustering data. A model is a statistical method and data is assumed to be generated by a distribution model with its parameter, θ . There may be many distributions involved and that is why it is called as mixture model. Since, Gaussian are normally assumed for data, this mixture model is categorized as Gaussian Mixture Model (GMM).

The EM algorithm is one algorithm that is commonly used for estimating the MLE in the presence of latent or missing variables. What is a latent variable? Let us assume that the dataset includes weights of boys and girls. Considering the fact that the boys' weights would be slightly more than the weights of the girls, one can assume that the larger weights are generated by one gaussian distribution with one set of parameters while girls' weights are generated with another set of parameters. There is an influence of gender in the data, but it is not directly present or observable. These are called latent variables. The EM algorithm is effective for estimating the PDF in the presence of latent variables.

Generally, there can be many unspecified distributions with different set of parameters. The EM algorithm has two stages:

1. Expectation (E) Stage – In this stage, the expected PDF and its parameters are estimated for each latent variable.
2. Maximization (M) stage – In this, the parameters are optimized using the MLE function.

This process is iterative, and the iteration is continued till all the latent variables are fitted by probability distributions effectively along with the parameters.

Non-parametric Density Estimation A non-parametric estimation can be generative or discriminative. Parzen window is a generative estimation method that finds $p(x | \Theta)$ as conditional density. Discriminative methods directly compute $p(\Theta | x)$ as posteriori probability. Parzen window and k -Nearest Neighbour (KNN) rule are examples of non-parametric density estimation. Let us discuss about them now.

Parzen Window Let there be ' n ' samples, $X = \{x_1, x_2, \dots, x_n\}$

The samples are drawn independently, called as identically independent distribution. Let R be the region that covers ' k ' samples of total ' n ' samples. Then, the probability density function is given as:

$$p = k/n \quad (2.38)$$

The estimate is given as:

$$p(x) = \frac{k/n}{V} \quad (2.39)$$

where, V is the volume of the region R . If R is the hypercube centered at x and h is the length of the hypercube, the volume V is h^2 for 2D square cube and h^3 for 3D cube.

The Parzen window is given as follows:

$$\varphi\left(\frac{x_i - x}{h}\right) = \begin{cases} 1 & \text{if } \frac{|x_i - x|}{h} < \frac{1}{2} \\ 0 & \text{otherwise} \end{cases} \quad (2.40)$$

The window indicates if the sample is inside the region or not. The Parzen probability density function estimate using Eq. (2.40) is given as:

$$\begin{aligned} p(x) &= \frac{k/n}{V} \\ &= \frac{1}{n} \sum_{i=1}^n \frac{1}{V_n} \varphi\left(\frac{x_i - x}{h}\right) \end{aligned} \quad (2.41)$$

This window can be replaced by any other function too. If Gaussian function is used, then it is called Gaussian density function.

KNN Estimation The KNN estimation is another non-parametric density estimation method. Here, the initial parameter k is determined and based on that k -neighbours are determined. The probability density function estimate is the average of the values that are returned by the neighbours.

2.10 FEATURE ENGINEERING AND DIMENSIONALITY REDUCTION TECHNIQUES

Features are attributes. Feature engineering is about determining the subset of features that form an important part of the input that improves the performance of the model, be it classification or any other model in machine learning.

Feature engineering deals with two problems – Feature Transformation and Feature Selection. Feature transformation is extraction of features and creating new features that may be helpful in increasing performance. For example, the height and weight may give a new attribute called Body Mass Index (BMI).

Feature subset selection is another important aspect of feature engineering that focuses on selection of features to reduce the time but not at the cost of reliability.

The subset selection reduces the dataset size by removing irrelevant features and constructs a minimum set of attributes for machine learning. If the dataset has n attributes, then time complexity is extremely high as n dimensions need to be processed for the given dataset. For n attributes, there are 2^n possible subsets. If the value of n is high, the problem becomes intractable. This is called 'curse of dimensionality'. Since, as the number of dimensions increases, the time complexity increases. The remedy is that some of the components that do not contribute much can be deleted. This results in the reduction of dimensionality. Choosing optimal attributes becomes a graph search problem. Typically, the feature subset selection problem uses greedy approach by looking for the best choice at the time using locally optimal choice while hoping that it would lead to global optimal solutions.

The features can be removed based on two aspects:

1. **Feature relevancy** – Some features contribute more for classification than other features. For example, a mole on the face can help in face detection than common features like nose. In simple words, the features should be relevant. The relevancy of the features can be determined based on information measures such as mutual information, correlation-based features like correlation coefficient and distance measures. Distance measures are discussed in Chapter 13 of this book.
2. **Feature redundancy** – Some features are redundant. For example, when a database table has a field called Date of birth, then age field is not relevant as age can be computed easily from date of birth. This helps in removing the column age that leads to reduction of dimension one.

So, the procedure is:

1. Generate all possible subsets
2. Evaluate the subsets and model performance
3. Evaluate the results for optimal feature selection

Filter-based selection uses statistical measures for assessing features. In this approach, no learning algorithm is used. Correlation and information gain measures like mutual information and entropy are all examples of this approach.

Wrapper-based methods use classifiers to identify the best features. These are selected and evaluated by the learning algorithms. This procedure is computationally intensive but has superior performance.

Let us discuss some of the important algorithms that fall under this category.

2.10.1 Stepwise Forward Selection

This procedure starts with an empty set of attributes. Every time, an attribute is tested for statistical significance for best quality and is added to the reduced set. This process is continued till a good reduced set of attributes is obtained.

2.10.2 Stepwise Backward Elimination

This procedure starts with a complete set of attributes. At every stage, the procedure removes the worst attribute from the set, leading to the reduced set.

Combined Approach Both forward and reverse methods can be combined so that the procedure can add the best attribute and remove the worst attribute.

2.10.3 Principal Component Analysis

The idea of the principal component analysis (PCA) or KL transform is to transform a given set of measurements to a new set of features so that the features exhibit high information packing properties. This leads to a reduced and compact set of features. Basically, this elimination is made possible because of the information redundancies. This compact representation is of a reduced dimension.

Consider a group of random vectors of the form:

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

The *mean vector* of the set of random vectors is defined as:

$$m_x = E\{x\}$$

The operator E refers to the expected value of the population. This is calculated theoretically using the probability density functions (PDF) of the elements x_i and the joint probability density functions between the elements x_i and x_j . From this, the covariance matrix can be calculated as:

$$C = E\{(x - m_x)(x - m_x)^T\} \quad (2.52)$$

For M random vectors, when M is large enough, the mean vector and covariance matrix can be approximately calculated as:

$$m_x = \frac{1}{M} \sum_{k=1}^M x_k \quad (2.53)$$

$$A = \frac{1}{M} \sum_{k=1}^M x_k x_k^T - m_x m_x^T \quad (2.54)$$

This covariance matrix is real and symmetric. If e_i and λ_i (where, $i = 1, 2, \dots, n$) be the set of eigen vectors and corresponding eigen values of the covariance matrix, the eigen values can be arranged in a descending order so that $\lambda_i \geq \lambda_{i+1}$ for $i = 1, 2, \dots, n - 1$. The corresponding eigen vectors are calculated. Based on this, the transform kernel is constructed. Let the transform kernel be A . Then, the matrix rows are formed from the eigen vectors of the covariance matrix.

The mapping of the vectors x to y using the transformation can now be described as:

$$y = A(x - m_x) \quad (2.55)$$

This transform is also called as Karhunen-Loeve or Hotelling transform. The original vector x can now be reconstructed as follows:

$$x = A^T y + m_x \quad (2.56)$$

The goal of PCA is to reduce the set of attributes to a newer, smaller set that captures the variance of the data. The variance is captured by fewer components, which would give the same result as the original, with all the attributes. Here, instead of using all the eigen vectors of the covariance matrix, only a small set that exhibits the variance can be used. By using a small set, the KL transform can achieve maximum compression of the available data.

If K largest eigen values are used, the recovered information would be:

$$x = A_K^T y + m_x \quad (2.57)$$

The advantages of PCA are immense. It reduces the attribute list by eliminating all irrelevant attributes. The PCA algorithm is as follows:

1. The target dataset x is obtained
2. The mean is subtracted from the dataset. Let the mean be m . Thus, the adjusted dataset is $X - m$. The objective of this process is to transform the dataset with zero mean.
3. The covariance of dataset x is obtained. Let it be C .

4. Eigen values and eigen vectors of the covariance matrix are calculated.
5. The eigen vector of the highest eigen value is the principal component of the dataset. The eigen values are arranged in a descending order. The feature vector is formed with these eigen vectors in its columns.

Feature vector = {eigen vector₁, eigen vector₂, ..., eigen vector_n}

6. Obtain the transpose of feature vector. Let it be A .
7. PCA transform is $y = A \times (x - m)$, where x is the input dataset, m is the mean, and A is the transpose of the feature vector.

The original data can be retrieved using the formula given below:

$$\text{Original data } (f) = \{(A)^{-1} \times y\} + m \quad (2.58)$$

$$= \{(A)^T \times y\} + m \quad (2.59)$$

The new data is a dimensionally reduced matrix that represents the original data. Therefore, PCA is effective in removing the attributes that do not contribute. If the original data is required, it can be obtained with no loss of information. Scree plot is a visualization technique to visualize the principal components or variables that play a more important role as compared to other attributes. Scree plot is a visualization technique to visualize the principal components visually. For a randomly selected dataset of 246 attributes, PCA is applied and its scree plot is shown in Figure 2.15. The scree plot indicates that only 6 out of 246 attributes are important.

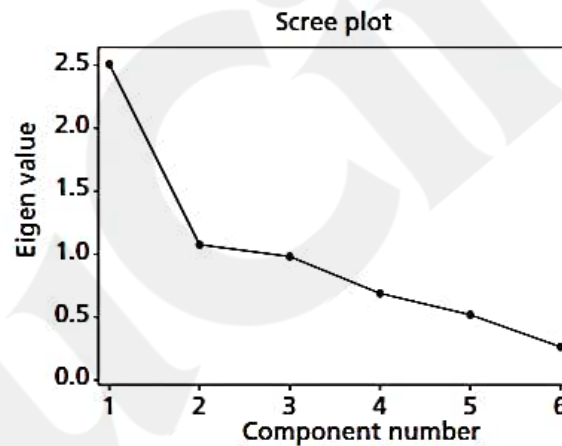


Figure 2.15: Scree Plot

From Figure 2.15, one can infer the relevance of the attributes. The scree plot indicates that the first attribute is more important than all other attributes.

Example 2.12: Let the data points be $\begin{pmatrix} 2 \\ 6 \end{pmatrix}$ and $\begin{pmatrix} 1 \\ 7 \end{pmatrix}$. Apply PCA and find the transformed data.

Again, apply the inverse and prove that PCA works.

Solution: One can combine two vectors into a matrix as follows:

The mean vector can be computed as Eq. (2.53) as follows:

$$\mu = \begin{pmatrix} \frac{2+1}{2} \\ \frac{6+7}{2} \end{pmatrix} = \begin{pmatrix} 1.5 \\ 6.5 \end{pmatrix}$$

As part of PCA, the mean must be subtracted from the data to get the adjusted data:

$$\begin{aligned}x_1 &= \begin{pmatrix} 2 - 1.5 \\ 6 - 6.5 \end{pmatrix} = \begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix} \\x_2 &= \begin{pmatrix} 1 - 1.5 \\ 7 - 6.5 \end{pmatrix} = \begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix}\end{aligned}$$

One can find the covariance for these data vectors. The covariance can be obtained using Eq. (2.54):

$$\begin{aligned}m_1 &= \begin{pmatrix} 0.5 \\ -0.5 \end{pmatrix} \begin{pmatrix} 0.5 & -0.5 \end{pmatrix} = \begin{pmatrix} 0.25 & -0.25 \\ -0.25 & 0.25 \end{pmatrix} \\m_2 &= \begin{pmatrix} -0.5 \\ 0.5 \end{pmatrix} \begin{pmatrix} -0.5 & 0.5 \end{pmatrix} = \begin{pmatrix} 0.25 & -0.25 \\ -0.25 & 0.25 \end{pmatrix}\end{aligned}$$

The final covariance matrix is obtained by adding these two matrices as:

$$C = \begin{pmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{pmatrix}$$

The eigen values and eigen vectors of matrix C can be obtained (left as an exercise) as $\lambda_1 = 1$, $\lambda_2 = 0$. The eigen vectors are $\begin{pmatrix} -1 \\ 1 \end{pmatrix}$ and $\begin{pmatrix} 1 \\ 1 \end{pmatrix}$. The matrix A can be obtained by packing the eigen vector of these eigen values (after sorting it) of matrix C . For this problem, $A = \begin{pmatrix} -1 & 1 \\ 1 & 1 \end{pmatrix}$. The transpose of A , $A^T = \begin{pmatrix} -1 & 1 \\ 1 & 1 \end{pmatrix}$ is also the same matrix as it is an orthogonal matrix. The matrix can be normalized by dividing each elements of the vector, by the norm of the vector to get:

$$A = \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix}$$

One can check that the PCA matrix A is orthogonal. A matrix is orthogonal is $A^{-1} = A$ and $AA^{-1} = I$.

$$\begin{aligned}AA^T &= \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \\&= \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}\end{aligned}$$

The transformed matrix y using Eq. (2.55) is given as:

$$y = A \times (x - m)$$

Recollect that $(x-m)$ is the adjusted matrix.

$$\begin{aligned}
 y &= A(x-m) = \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{pmatrix} \\
 &= \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} \frac{1}{2} & -\frac{1}{2} \\ -\frac{1}{2} & \frac{1}{2} \end{pmatrix} \left(\text{for convenience } 0.5 = \frac{1}{2} \right) \\
 &= \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & 0 \end{pmatrix}
 \end{aligned}$$

One can check the original matrix can be retrieved from this matrix as:

$$\begin{aligned}
 x &= A^T y + m = \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \begin{pmatrix} -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 1.5 \\ 6.5 \end{pmatrix} \\
 &= \begin{pmatrix} \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} \end{pmatrix} + \begin{pmatrix} 1.5 \\ 6.5 \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 6 & 7 \end{pmatrix}
 \end{aligned}$$

Therefore, one can infer the original is obtained without any loss of information.

2.10.4 Linear Discriminant Analysis

Linear Discriminant Analysis (LDA) is also a feature reduction technique like PCA. The focus of LDA is to project higher dimension data to a line (lower dimension data). LDA is also used to classify the data. Let there be two classes, c_1 and c_2 . Let μ_1 and μ_2 be the mean of the patterns of two classes. The mean of the class c_1 and c_2 can be computed as:

$$\mu_1 = \frac{1}{N_1} \sum_{x_i \in c_1} x_i \text{ and } \mu_2 = \frac{1}{N_2} \sum_{x_i \in c_2} x_i$$

The aim of LDA is to optimize the function:

$$J(V) = \frac{V^T \sigma_B V}{V^T \sigma_W V} \quad (2.60)$$

where, V is the linear projection and σ_B and σ_W are class scatter matrix and within scatter matrix, respectively. For the two-class problem, these matrices are given as:

$$\sigma_B = N_1(\mu_1 - \mu)(\mu_1 - \mu)^T + N_2(\mu_2 - \mu)(\mu_2 - \mu)^T \quad (2.61)$$

$$\sigma_W = \sum_{x_i \in c_1} (x_i - \mu_1)(x_i - \mu_1)^T + \sum_{x_i \in c_2} (x_i - \mu_2)(x_i - \mu_2)^T \quad (2.62)$$

The maximization of $J(V)$ should satisfy the equation:

$$\sigma_B V = \lambda \sigma_W V \text{ or } \sigma_W^{-1} \sigma_B V = \lambda V \quad (2.63)$$

As $\sigma_B V$ is always in the direction of $(\mu_1 - \mu_2)$, V can be given as:

$$V = \sigma_W^{-1}(\mu_1 - \mu_2) \quad (2.64)$$

Let $V = \{v_1, v_2, \dots, v_d\}$ be the generalized eigen vectors of σ_B and σ_W , where, d is the largest eigen values as in PCA. The transformation of x is then given as:

$$y = V_d^T x \quad (2.65)$$

Like in PCA, the largest eigen values can be retained to have projections.

2.10.5 Singular Value Decomposition

Singular Value Decomposition (SVD) is another useful decomposition technique. Let A be the matrix, then the matrix A can be decomposed as:

$$A = USV^T \quad (2.66)$$

Here, A is the given matrix of dimension $m \times n$, U is the orthogonal matrix whose dimension is $m \times m$, S is the diagonal matrix of dimension $n \times n$, and V is the orthogonal matrix. The procedure for finding decomposition matrix is given as follows:

1. For a given matrix, find AA^T
2. Find eigen values of AA^T
3. Sort the eigen values in a descending order. Pack the eigen vectors as a matrix U .
4. Arrange the square root of the eigen values in diagonal. This matrix is diagonal matrix, S .
5. Find eigen values and eigen vectors for $A^T A$. Find the eigen value and pack the eigen vector as a matrix called V .

Thus, $A = USV^T$. Here, U and V are orthogonal matrices. The columns of U and V are left and right singular values, respectively. SVD is useful in compression, as one can decide to retain only a certain component instead of the original matrix A as:

$$a_{ij} = \sum_{k=1}^n u_{ik} s_k v_{jk} \quad (2.67)$$

Based on the choice of retention, the compression can be controlled.

Example 2.13: Find SVD of the matrix:

$$A = \begin{pmatrix} 1 & 2 \\ 4 & 9 \end{pmatrix}$$

Solution: The first step is to compute:

$$AA^T = \begin{pmatrix} 1 & 2 \\ 4 & 9 \end{pmatrix} \begin{pmatrix} 1 & 4 \\ 2 & 9 \end{pmatrix} = \begin{pmatrix} 5 & 22 \\ 22 & 97 \end{pmatrix}$$

The eigen value and eigen vector of this matrix can be calculated to get U . The eigen values of this matrix are 0.0098 and 101.9902.

The eigen vectors of this matrix are:

$$u_1 = \begin{pmatrix} 0.2268 \\ 1 \end{pmatrix}$$

$$u_2 = \begin{pmatrix} -4.4086 \\ 1 \end{pmatrix}$$

These vectors are normalized to get the vectors respectively as:

$$u_1 = \begin{pmatrix} 0.2212 \\ 0.9752 \end{pmatrix}$$

$$u_2 = \begin{pmatrix} -0.9752 \\ 0.2212 \end{pmatrix}$$

The matrix U can be obtained by concatenating the above vector as:

$$U = [u_1, u_2] = \begin{pmatrix} 0.2212 & -0.9752 \\ 0.9752 & 0.2212 \end{pmatrix}$$

The matrix V can be obtained by finding $A^T A$. It is $\begin{pmatrix} 17 & 38 \\ 38 & 85 \end{pmatrix}$. The eigen values are 0.0098 and 101.9902. The eigen vectors can be found as follows:

$$v_1 = \begin{pmatrix} 0.447 \\ 1 \end{pmatrix} \text{ when } \lambda = 101.99$$

$$v_2 = \begin{pmatrix} -2.236 \\ 1 \end{pmatrix} \text{ when } \lambda = 0.0098$$

The above can be normalized as follows:

$$v_1 = \begin{pmatrix} 0.4082 \\ 0.9129 \end{pmatrix}$$

$$v_2 = \begin{pmatrix} -0.9129 \\ 0.4082 \end{pmatrix}$$

The matrix V can be obtained by concatenating the above vector as:

$$V = [v_1, v_2] = \begin{pmatrix} 0.4081 & -0.9129 \\ 0.9129 & 0.4082 \end{pmatrix}$$

The matrix S can be found as the diagonal matrix as:

$$S = \begin{pmatrix} \sqrt{101.9902} & 0 \\ 0 & \sqrt{0.0098} \end{pmatrix} = \begin{pmatrix} 10.099 & 0 \\ 0 & 0.099 \end{pmatrix}$$

Therefore, the matrix decomposition $A = U S V^T$ is complete.

The main advantage of SVD is compression. A matrix, say an image, can be decomposed and selectively only certain components can be retained by making all other elements zero. This reduces the contents of image while retaining the quality of the image. SVD is useful in data reduction too.

2.2 KNOWING WHAT YOU KNOW: TESTING MACHINE LEARNING ALGORITHMS

The purpose of learning is to get better at predicting the outputs, be they class labels or continuous regression values. The only real way to know how successfully the algorithm has learnt is to compare the predictions with known target labels, which is how the training is done for supervised learning. This suggests that one thing you can do is just to look at the error that the algorithm makes on the training set.

However, we want the algorithms to generalise to examples that were not seen in the training set, and we obviously can't test this by using the training set. So we need some different data, a **test set**, to test it on as well. We use this test set of (input, target) pairs by feeding them into the network and comparing the predicted output with the target, but we don't modify the weights or other parameters for them: we use them to decide how well the algorithm has learnt. The only problem with this is that it reduces the amount of data that we have available for training, but that is something that we will just have to live with.

2.2.1 Overfitting

Unfortunately, things are a little bit more complicated than that, since we might also want to know how well the algorithm is generalising as it learns: we need to make sure that we do enough training that the algorithm generalises well. In fact, there is at least as much danger in over-training as there is in under-training. The number of degrees of variability in most machine learning algorithms is huge — for a neural network there are lots of weights, and each of them can vary. This is undoubtedly more variation than there is in the function we are learning, so we need to be careful: if we train for too long, then we will overfit the data, which means that we have learnt about the noise and inaccuracies in the data as well as the actual function. Therefore, the model that we learn will be much too complicated, and won't be able to generalise.

Figure 2.5 shows this by plotting the predictions of some algorithm (as the curve) at

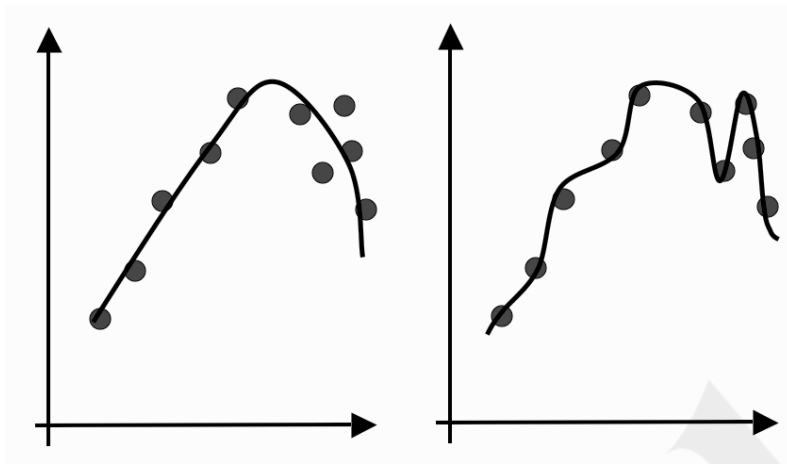


FIGURE 2.5 The effect of overfitting is that rather than finding the generating function (as shown on the left), the neural network matches the inputs perfectly, including the noise in them (on the right). This reduces the generalisation capabilities of the network.

two different points in the learning process. On the left of the figure the curve fits the overall trend of the data well (it has generalised to the underlying general function), but the training error would still not be that close to zero since it passes near, but not through, the training data. As the network continues to learn, it will eventually produce a much more complex model that has a lower training error (close to zero), meaning that it has memorised the training examples, including any noise component of them, so that it has overfitted the training data.

We want to stop the learning process before the algorithm overfits, which means that we need to know how well it is generalising at each timestep. We can't use the training data for this, because we wouldn't detect overfitting, but we can't use the testing data either, because we're saving that for the final tests. So we need a third set of data to use for this purpose, which is called the **validation set** because we're using it to validate the learning so far. This is known as **cross-validation** in statistics. It is part of **model selection**: choosing the right parameters for the model so that it generalises as well as possible.

2.2.2 Training, Testing, and Validation Sets

We now need three sets of data: the **training set** to actually train the algorithm, the **validation set** to keep track of how well it is doing as it learns, and the **test set** to produce the final results. This is becoming expensive in data, especially since for supervised learning it all has to have target values attached (and even for unsupervised learning, the validation and test sets need targets so that you have something to compare to), and it is not always easy to get accurate labels (which may well be why you want to learn about the data). The area of **semi-supervised learning** attempts to deal with this need for large amounts of labelled data; see the Further Reading section for some references.

Clearly, each algorithm is going to need some reasonable amount of data to learn from (precise needs vary, but the more data the algorithm sees, the more likely it is to have seen examples of each possible type of input, although more data also increases the computational time to learn). However, the same argument can be used to argue that the validation and

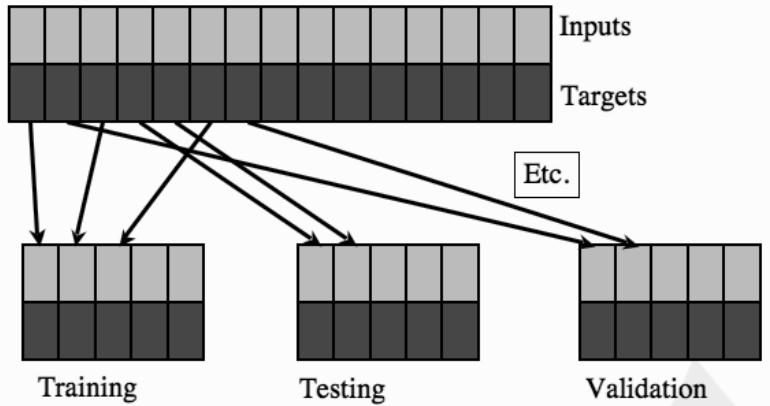


FIGURE 2.6 The dataset is split into different sets, some for training, some for validation, and some for testing.

test sets should also be reasonably large. Generally, the exact proportion of training to testing to validation data is up to you, but it is typical to do something like 50:25:25 if you have plenty of data, and 60:20:20 if you don't. How you do the splitting can also matter. Many datasets are presented with the first set of datapoints being in class 1, the next in class 2, and so on. If you pick the first few points to be the training set, the next the test set, etc., then the results are going to be pretty bad, since the training did not see all the classes. This can be dealt with by randomly reordering the data first, or by assigning each datapoint randomly to one of the sets, as is shown in Figure 2.6.

If you are really short of training data, so that if you have a separate validation set there is a worry that the algorithm won't be sufficiently trained; then it is possible to perform **leave-some-out**, **multi-fold cross-validation**. The idea is shown in Figure 2.7. The dataset is randomly partitioned into K subsets, and one subset is used as a validation set, while the algorithm is trained on all of the others. A different subset is then left out and a new model is trained on that subset, repeating the same process for all of the different subsets. Finally, the model that produced the lowest validation error is tested and used. We've traded off data for computation time, since we've had to train K different models instead of just one. In the most extreme case of this there is **leave-one-out cross-validation**, where the algorithm is validated on just one piece of data, training on all of the rest.

2.2.3 The Confusion Matrix

Regardless of how much data we use to test the trained algorithm, we still need to work out whether or not the result is good. We will look here at a method that is suitable for classification problems that is known as the **confusion matrix**. For regression problems things are more complicated because the results are continuous, and so the most common thing to use is the sum-of-squares error that we will use to drive the training in the following chapters. We will see these methods being used as we look at examples.

The confusion matrix is a nice simple idea: make a square matrix that contains all the possible classes in both the horizontal and vertical directions and list the classes along the top of a table as the predicted outputs, and then down the left-hand side as the targets. So for example, the element of the matrix at (i, j) tells us how many input patterns were put

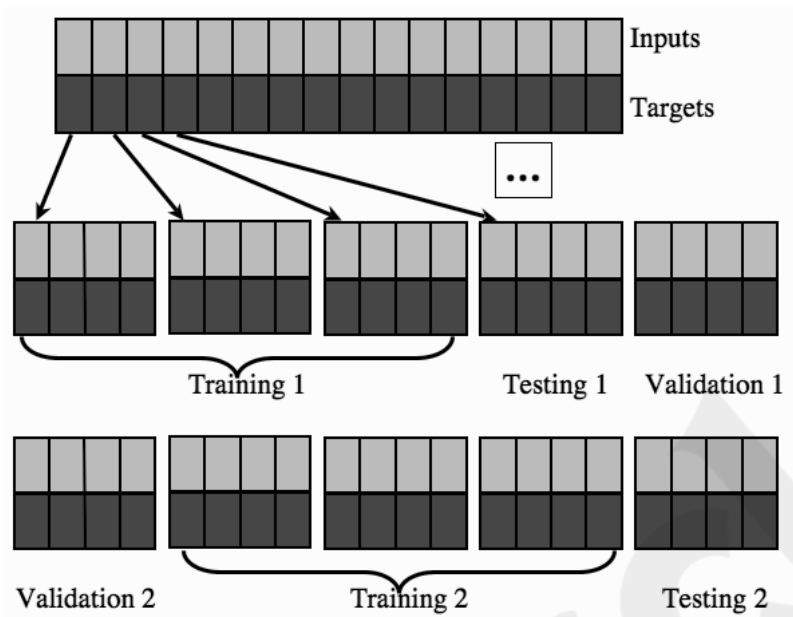


FIGURE 2.7 Leave-some-out, multi-fold cross-validation gets around the problem of data shortage by training many models. It works by splitting the data into sets, training a model on most sets and holding one out for validation (and another for testing). Different models are trained with different sets being held out.

into class i in the targets, but class j by the algorithm. Anything on the leading diagonal (the diagonal that starts at the top left of the matrix and runs down to the bottom right) is a correct answer. Suppose that we have three classes: C_1, C_2 , and C_3 . Now we count the number of times that the output was class C_1 when the target was C_1 , then when the target was C_2 , and so on until we’ve filled in the table:

	Outputs		
	C_1	C_2	C_3
C_1	5	1	0
C_2	1	4	1
C_3	2	0	4

This table tells us that, for the three classes, most examples were classified correctly, but two examples of class C_3 were misclassified as C_1 , and so on. For a small number of classes this is a nice way to look at the outputs. If you just want one number, then it is possible to divide the sum of the elements on the leading diagonal by the sum of all of the elements in the matrix, which gives the fraction of correct responses. This is known as the **accuracy**, and we are about to see that it is not the last word in evaluating the results of a machine learning algorithm.

2.2.4 Accuracy Metrics

We can do more to analyse the results than just measuring the **accuracy**. If you consider the possible outputs of the classes, then they can be arranged in a simple chart like this

(where a **true positive** is an observation correctly put into class 1, while a **false positive** is an observation incorrectly put into class 1, while negative examples (both true and false) are those put into class 2):

True Positives	False Positives
False Negatives	True Negatives

The entries on the leading diagonal of this chart are correct and those off the diagonal are wrong, just as with the confusion matrix. Note, however, that this chart and the concepts of false positives, etc., are based on binary classification.

Accuracy is then defined as the sum of the number of true positives and true negatives divided by the total number of examples (where # means ‘number of’, and TP stands for True Positive, etc.):

$$\text{Accuracy} = \frac{\#TP + \#FP}{\#TP + \#FP + \#TN + \#FN}. \quad (2.2)$$

The problem with accuracy is that it doesn’t tell us everything about the results, since it turns four numbers into just one. There are two complementary pairs of measurements that can help us to interpret the performance of a classifier, namely **sensitivity** and **specificity**, and **precision** and **recall**. Their definitions are shown next, followed by some explanation.

$$\text{Sensitivity} = \frac{\#TP}{\#TP + \#FN} \quad (2.3)$$

$$\text{Specificity} = \frac{\#TN}{\#TN + \#FP} \quad (2.4)$$

$$\text{Precision} = \frac{\#TP}{\#TP + \#FP} \quad (2.5)$$

$$\text{Recall} = \frac{\#TP}{\#TP + \#FN} \quad (2.6)$$

Sensitivity (also known as the **true positive rate**) is the ratio of the number of correct positive examples to the number classified as positive, while specificity is the same ratio for negative examples. Precision is the ratio of correct positive examples to the number of actual positive examples, while recall is the ratio of the number of correct positive examples out of those that were classified as positive, which is the same as sensitivity. If you look at the chart again you can see that sensitivity and specificity sum the columns for the denominator, while precision and recall sum the first column and the first row, and so miss out some information about how well the learner does on the negative examples.

Together, either of these pairs of measures gives more information than just the accuracy. If you consider precision and recall, then you can see that they are to some extent inversely related, in that if the number of false positives increases (meaning that the algorithm is using a broader definition of that class), then the number of false negatives often decreases, and vice versa. They can be combined to give a single measure, the F_1 measure, which can be written in terms of precision and recall as:

$$F_1 = 2 \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \quad (2.7)$$

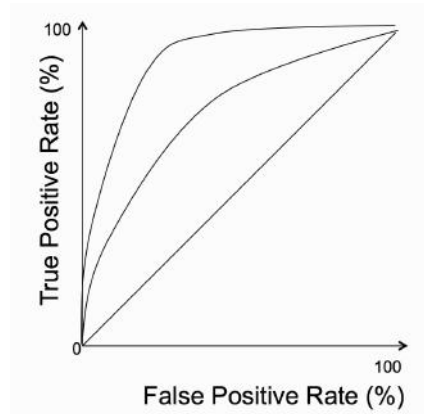


FIGURE 2.8 An example of an ROC curve. The diagonal line represents exactly chance, so anything above the line is better than chance, and the further from the line, the better. Of the two curves shown, the one that is further away from the diagonal line would represent a more accurate method.

and in terms of the numbers of false positives, etc. (from which it can be seen that it computes the mean of the false examples) as:

$$F_1 = \frac{\#TP}{\#TP + (\#FN + \#FP)/2}. \quad (2.8)$$

2.2.5 The Receiver Operator Characteristic (ROC) Curve

Since we can use these measures to evaluate a particular classifier, we can also compare classifiers – either the same classifier with different learning parameters, or completely different classifiers. In this case, the Receiver Operator Characteristic curve (almost always known just as the ROC curve) is useful. This is a plot of the percentage of true positives on the y axis against false positives on the x axis; an example is shown in Figure 2.8. A single run of a classifier produces a single point on the ROC plot, and a perfect classifier would be a point at $(0, 1)$ (100% true positives, 0% false positives), while the anti-classifier that got everything wrong would be at $(1, 0)$; so the closer to the top-left-hand corner the result of a classifier is, the better the classifier has performed. Any classifier that sits on the diagonal line from $(0, 0)$ to $(1, 1)$ behaves exactly at the chance level (assuming that the positive and negative classes are equally common) and so presumably a lot of learning effort is wasted since a fair coin would do just as well.

In order to compare classifiers, or choices of parameters settings for the same classifier, you could just compute the point that is furthest from the ‘chance’ line along the diagonal. However, it is normal to compute the area under the curve (AUC) instead. If you only have one point for each classifier, the curve is the trapezoid that runs from $(0, 0)$ up to the point and then from there to $(1, 1)$. If there are more points (based on more runs of the classifier, such as trained and/or tested on different datasets), then they are just included in order along the diagonal line.

The key to getting a curve rather than a point on the ROC curve is to use cross-validation. If you use 10-fold cross-validation, then you have 10 classifiers, with 10 different

test sets, and you also have the ‘ground truth’ labels. The true labels can be used to produce a ranked list of the different cross-validation-trained results, which can be used to specify a curve through the 10 datapoints on the ROC curve that correspond to the results of this classifier. By producing an ROC curve for each classifier it is possible to compare their results.

2.2.6 Unbalanced Datasets

Note that for the accuracy we have implicitly assumed that there are the same number of positive and negative examples in the dataset (which is known as a **balanced** dataset). However, this is often not true (this can potentially cause problems for the learners as well, as we shall see later in the book). In the case where it is not, we can compute the **balanced accuracy** as the sum of sensitivity and specificity divided by 2. However, a more correct measure is **Matthew’s Correlation Coefficient**, which is computed as:

$$MCC = \frac{\#TP \times \#TN - \#FP \times \#FN}{\sqrt{(\#TP + \#FP)(\#TP + \#FN)(\#TN + \#FP)(\#TN + \#FN)}} \quad (2.9)$$

If any of the brackets in the denominator are 0, then the whole of the denominator is set to 1. This provides a balanced accuracy computation.

As a final note on these methods of evaluation, if there are more than two classes and it is useful to distinguish the different types of error, then the calculations get a little more complicated, since instead of one set of false positives and one set of false negatives, you have some for each class. In this case, specificity and recall are not the same. However, it is possible to create a set of results, where you use one class as the positives and everything else as the negatives, and repeat this for each of the different classes.

2.2.7 Measurement Precision

There is a different way to evaluate the accuracy of a learning system, which unfortunately also uses the word **precision**, although with a different meaning. The concept here is to treat the machine learning algorithm as a measurement system. We feed in inputs and look at the outputs that we get. Even before comparing them to the target values, we can measure something about the algorithm: if we feed in a set of similar inputs, then we would expect to get similar outputs for them. This measure of the variability of the algorithm is also known as **precision**, and it tells us how repeatable the predictions that the algorithm makes are. It might be useful to think of precision as being something like the variance of a probability distribution: it tells you how much spread around the mean to expect.

The point is that just because an algorithm is precise it does not mean that it is accurate – it can be precisely wrong if it always gives the wrong prediction. One measure of how well the algorithm’s predictions match reality is known as **trueness**, and it can be defined as the average distance between the correct output and the prediction. Trueness doesn’t usually make much sense for classification problems unless there is some concept of certain classes being similar to each other. Figure 2.9 illustrates the idea of trueness and precision in the traditional way: as a darts game, with four examples with varying trueness and precision for the three darts thrown by a player.

This section has considered the endpoint of machine learning, looking at the outputs, and thinking about what we need to do with the input data in terms of having multiple datasets, etc. In the next section we return to the starting point and consider how we can start analysing a dataset by dealing with probabilities.

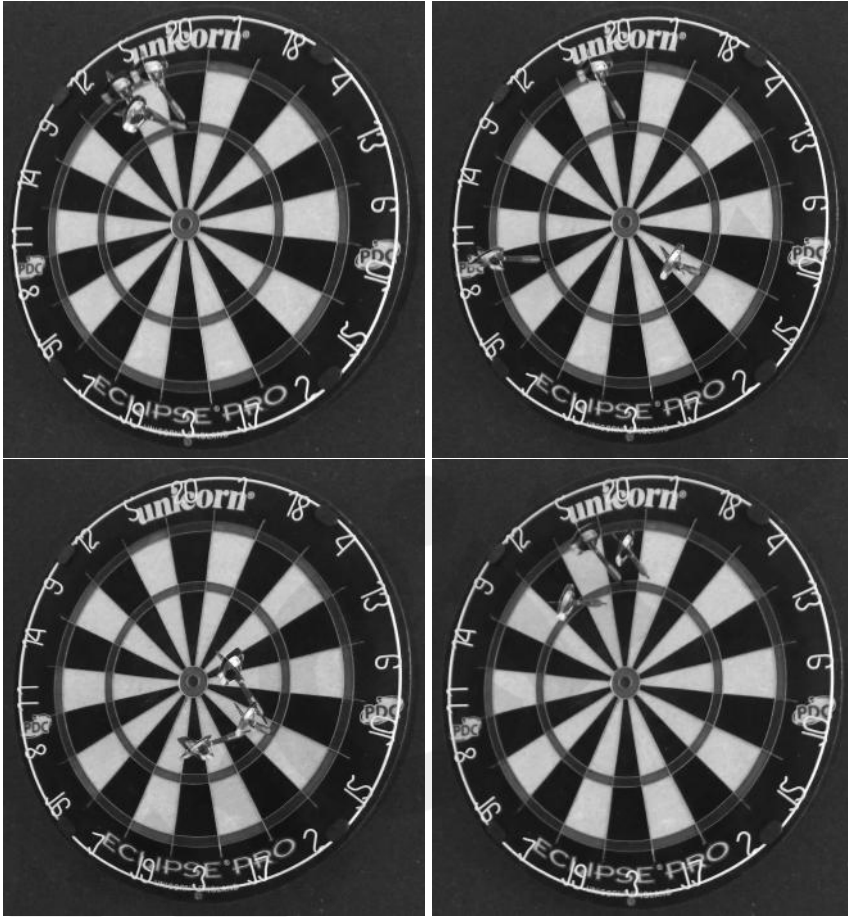


FIGURE 2.9 Assuming that the player was aiming for the highest-scoring triple 20 in darts (the segments each score the number they are labelled with, the narrow band on the outside of the circle scores double and the narrow band halfway in scores triple; the outer and inner 'bullseye' at the centre score 25 and 50, respectively), these four pictures show different outcomes. *Top left*: very accurate: high precision and trueness, *top right*: low precision, but good trueness, *bottom left*: high precision, but low trueness, and *bottom right*: reasonable trueness and precision, but the actual outputs are not very good. (Thanks to Stefan Nowicki, whose dartboard was used for these pictures.)

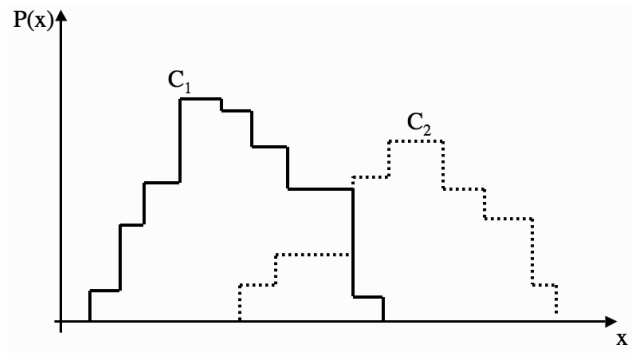


FIGURE 2.10 A histogram of feature values (x) against their probability for two classes.